

GUÍA DE DESPLIEGUE Y OPERACIONES

1. Introducción

La presente guía describe el proceso de despliegue, configuración, ejecución y operación del sistema de gestión retail desarrollado bajo arquitectura hexagonal. El objetivo es permitir la instalación y ejecución del sistema de forma local o remota utilizando contenedores Docker, garantizando un entorno estable para pruebas, desarrollo y demostración académica.

Esta guía incluye:

- Requisitos previos.
 - Instalación de dependencias.
 - Configuración del entorno.
 - Ejecución mediante Docker.
 - Operaciones básicas del sistema.
 - Gestión de base de datos.
 - Monitoreo básico.
 - Procedimientos de respaldo y recuperación.
 - Solución de errores comunes.
-

2. Objetivo de la Guía

Objetivo General

Definir el procedimiento técnico necesario para desplegar y operar el sistema retail de manera segura, organizada y reproducible.

Objetivos Específicos

- Facilitar la instalación del sistema.
 - Garantizar la correcta configuración del entorno.
 - Permitir la ejecución del sistema mediante Docker.
 - Establecer procedimientos operativos básicos.
 - Definir mecanismos de respaldo y recuperación.
 - Proporcionar soporte ante errores frecuentes.
-

3. Requisitos del Sistema

Requisitos de Hardware

Recurso	Recomendado
Procesador	Intel i5 o Ryzen 5
Memoria RAM	8 GB
Almacenamiento	20 GB libres
Conexión	Internet estable

Requisitos de Software

Software	Versión
Docker Desktop	Última estable
Docker Compose	2.x
Git	Última estable
Node.js	20.x
MySQL	8.x
Redis	7.x

4. Arquitectura de Despliegue

El sistema se desplegará utilizando contenedores Docker independientes para garantizar aislamiento y facilidad de configuración.

Componentes del Sistema

Frontend

- Aplicación React + Vite.
- Puerto: 5173.

Backend

- API REST desarrollada en NestJS.
- Puerto: 3000.

Base de Datos

- MySQL 8.
- Puerto: 3306.

Caché y Sesiones

- Redis 7.
- Puerto: 6379.

5. Estructura del Proyecto

retail-system/

|

└─ backend/

└─ frontend/

└─ docs/

└─ docker-compose.yml

└─ .env

└─ .env.example

└─ README.md

6. Instalación del Sistema

Paso 1: Clonar el Repositorio

```
git clone https://github.com/tuusuario/retail-system.git
```

```
cd retail-system
```

Paso 2: Configurar Variables de Entorno

Copiar el archivo de ejemplo:

```
cp .env.example .env
```

Editar el archivo .env:

```
DATABASE_URL=mysql://root:root@mysql:3306/retail_db
```

JWT_SECRET=secret_demo

REDIS_HOST=redis

REDIS_PORT=6379

PORT=3000

Paso 3: Levantar Contenedores Docker

docker-compose up -d

Este comando iniciará:

- Backend NestJS.
- Frontend React.
- MySQL.
- Redis.

Paso 4: Verificar Contenedores Activos

docker ps

Los servicios deben aparecer con estado Up.

7. Configuración de Base de Datos

Ejecutar Migraciones Prisma

docker-compose exec api npx prisma migrate deploy

Generar Cliente Prisma

docker-compose exec api npx prisma generate

Sembrar Datos de Prueba

docker-compose exec api npx ts-node scripts/seed.ts

Esto insertará:

- Usuarios demo.
 - Productos demo.
 - Clientes demo.
 - Categorías demo.
-

8. Acceso al Sistema

URLs Locales

Servicio	URL
Frontend	http://localhost:5173
Backend API	http://localhost:3000
Swagger	http://localhost:3000/api/docs

9. Usuarios de Prueba

Administrador

Campo	Valor
Usuario	admin@demo.com
Contraseña	admin123

Vendedor

Campo	Valor
Usuario	vendedor@demo.com
Contraseña	vendedor123

10. Operaciones Básicas del Sistema

Inicio del Sistema

docker-compose up -d

Detener el Sistema

docker-compose down

Reiniciar Servicios

docker-compose restart

Ver Logs del Sistema

Backend

docker-compose logs api

Frontend

docker-compose logs frontend

Base de Datos

docker-compose logs mysql

11. Flujo Operativo del Sistema

Flujo de Inicio de Sesión

1. El usuario ingresa correo y contraseña.
 2. El backend valida credenciales.
 3. Se genera token JWT.
 4. El frontend almacena sesión.
-

Flujo de Registro de Venta

1. Seleccionar productos.
2. Agregar productos al carrito.
3. Calcular subtotal e IGV.

4. Registrar venta.
 5. Descontar stock automáticamente.
 6. Generar comprobante PDF.
-

Flujo de Caja

Apertura

- Registrar monto inicial.

Cierre

- Registrar monto contado.
 - Comparar con monto esperado.
-

12. Documentación API

La documentación REST estará disponible mediante Swagger.

Acceso Swagger

<http://localhost:3000/api/docs>

La documentación incluirá:

- Endpoints.
 - Métodos HTTP.
 - Parámetros.
 - Respuestas JSON.
 - Autenticación JWT.
-

13. Respaldo y Recuperación

Respaldo de Base de Datos

```
docker exec mysql_container \
```

```
mysqldump -u root -p retail_db > backup.sql
```

Restaurar Base de Datos

```
docker exec -i mysql_container \
mysql -u root -p retail_db < backup.sql
```

14. Monitoreo Básico

Verificar Uso de Contenedores

```
docker stats
```

Verificar Estado de Servicios

```
docker-compose ps
```

Verificar Espacio en Disco

```
docker system df
```

15. Seguridad Básica

Medidas Implementadas

- Autenticación JWT.
 - Contraseñas cifradas con bcrypt.
 - Variables sensibles en .env.
 - Separación de contenedores.
 - Validaciones backend.
-

16. Despliegue en VPS (Opcional)

Requisitos

- Ubuntu Server 22.04.
 - Docker instalado.
 - Docker Compose instalado.
-

Pasos

Actualizar Sistema

```
sudo apt update && sudo apt upgrade -y
```

Instalar Docker

```
curl -fsSL https://get.docker.com | sh
```

Clonar Proyecto

```
git clone https://github.com/tuusuario/retail-system.git
```

Levantar Servicios

```
docker-compose up -d
```

Acceso Remoto

```
http://IP_DEL_SERVIDOR:5173
```

17. Problemas Frecuentes y Soluciones

Error: Puerto ocupado

Solución

Verificar procesos:

```
netstat -ano
```

Cambiar puertos en docker-compose.yml.

Error: Prisma no conecta a MySQL

Solución

Verificar:

- Variables .env.
- Estado del contenedor MySQL.
- URL de conexión.

Error: Docker no inicia

Solución

Reiniciar Docker Desktop o ejecutar:

`docker-compose down`

`docker-compose up -d`

Error: Frontend no consume API

Solución

Verificar:

- URL del backend.
 - Variables de entorno.
 - Configuración CORS en NestJS.
-

18. Mantenimiento Básico

Actualizar Dependencias

Backend

`npm update`

Frontend

`npm update`

Limpiar Contenedores No Utilizados

`docker system prune -a`

19. Buenas Prácticas Operativas

- Mantener respaldos frecuentes.
- No modificar directamente la base de datos.
- Validar logs regularmente.

- Mantener variables sensibles fuera del repositorio.
 - Utilizar ramas Git para nuevas funcionalidades.
 - Probar cambios antes de despliegue final.
-

20. Conclusión

La presente guía permite desplegar y operar el sistema retail de forma organizada y reproducible mediante Docker. Gracias a la arquitectura hexagonal y al uso de contenedores, el sistema puede ejecutarse tanto en entornos locales como remotos, facilitando pruebas, mantenimiento y demostraciones académicas.

El despliegue propuesto garantiza:

- Facilidad de instalación.
- Separación de servicios.
- Escalabilidad futura.
- Estabilidad para demostraciones.
- Simplicidad de mantenimiento y operación.